



AMERICAN UNIVERSITY OF PHNOM PENH

STUDY LOCALLY. LIVE GLOBALLY.

Fall 2025 Semester

ICT 361 Introduction to Robotics

Face Following Robot Report

Instructor/Professor: Theara SENG

12th December 2025

Name: OUCH Phallyda

ID: 2024558

Major: Computer Science / ITM

Face-Following Robot Project Report

Abstract / Executive Summary

In this project, I will be working on a face-following robot project. A face-following robot is a robotic system that is capable of detecting human faces and 5 landmarks, such as both eyes, nose, and mouth corners within the camera frame. This project implements a real-time face-following robot using RetinaFace MobileNet0.25 model, a lightweight yet accurate face detection model that is suitable for this project. The model is exported to ONNX to be deployed on a Raspberry Pi 4 with ONNX Runtime for real-time detection.

The robot continuously detect faces, extracts key facial landmarks, and will be able to estimate the position and distance of the face in the camera frame and move accordingly to keep the selected face that is the biggest and closest to the camera at the center of the frame and at a safe distance. Real-world testing demonstrated stable tracking at 1.5 - 8 FPS, and robust detection in common indoor environments. The robot successfully tracks faces and outputs the corresponding movement of commands, but physical movement was not executed. Overall, the system demonstrates accurate face and landmark detection, with behavior consistent in indoor conditions, though the goal of full physical face-following was not fully achieved.

I. Introduction

Face detection and tracking are essential components in modern robotics, enabling robots to interact with humans even more easily and naturally. Traditional computer vision methods struggle with variations in lighting, pose, angle, and real-time performance on small, embedded devices. This motivates the use of AI and deep learning-based Computer Vision, which provides robust detection of accuracy even under challenging conditions.

The goal of this project is to develop a robot capable of detecting any human face with the 5 important landmarks and following the face that is the biggest and closest to the frame by only using a camera and an onboard AI system.

The main challenges included:

- Detecting faces in real time on a low-power device
- Handling changes in lighting, angles, and distances
- Ensuring stable robot movement without jitter
- Converting and optimizing models for embedded inference

Our approach uses RetinaFace MobileNet0.25, a lightweight and fast landmark-aware face detector. The model is converted to ONNX, optimized, and deployed using ONNX Runtime on Raspberry Pi 4, Custom tracking logic computes facial center offset and distance, which are translated into robot movement commands.

II. Model Architecture & Training

A. Pretrained Model Usage

For this project, I used the MobileNet0.25 RetinaFace model, which is used in its pretrained form. It was originally trained on the WIDER FACE dataset, which includes:

- Thousands of images with diverse face scales, poses, and occlusion conditions
- 5-point landmark annotations (left eye, right eyes, nose, right mouth corner, left mouth corner)

Since my own dataset that I trained only has bounding boxes and lacks the important 5 landmarks annotations, it cannot be used for this project. Pretraining allows the model to generalize new faces effectively. Therefore:

- No training or fine-tuning is required
- The model can be directly exported to ONNX for inference using the `export_onnx.py`
- Landmark detection is available immediately for robot tracking

B. Model overview

The project uses RetinaFace MobileNet0.25, a lightweight single stage face detector designed for real-time applications. RetinaFace is known for achieving high accuracy on face detection benchmarks due to two defining characteristics:

1. **Single stage Architecture:** Unlike two stage detectors, RetinaFace predicts bounding boxes, facial landmarks, and detection of confidence in a single forward pass, enabling faster inference.
2. **Landmark regression branch:** RetinaFace predicts five facial landmarks, such as left eye, right eye, nose, left mouth corner, and right mouth corner.

These landmarks significantly improve tracking stability by allowing the robot to determine not only the position but also the orientation of the face.

C. Architecture Breakdown

RetinaFace consists of several key components that are significant such as:

1. **Backbone:** This project relies on MobileNet0.25, which is extremely lightweight for CNN. The dataset has a 0.25 width multiplier that reduces computation cost and is designed for mobile and embedded systems. This dataset extracts multi-resolution feature maps used for small and large face detection.
2. **Feature Pyramid Network (FPN):** combines low-level and high-level features, allows detecting faces across a wide range of sizes, and is important for tracking multiple people who move closer or further away.
3. **SSH Context Module:** SSH or known as Single Stage Headless, enhances context by enlarging receptive fields without significantly increasing computation. This allows RetinaFace to detect partially occluded or angled faces more reliably.

4. **Output Heads:** for the classification head, we use the probability of face or no-face, while using x, y, w, h for the bounding box regression head, and 5 landmarks $x \times 2$ coordinates for the landmark regression head. These outputs feed directly into the robot's face tracking logic.

D. Training Details

Since our project uses an existing pre-trained model, the training process followed the original implementation by RetinaFace and biubug6 contributors. For the dataset, I use the WIDER FACE dataset that was provided in the GitHub README from biubug6 with 32 203 images and around 393 000 labeled faces in that dataset. It also includes easy, medium, and hard faces such as small, occluded, heavily angled, etc. Below is the Hyperparameters used in RetinaFace Training:

Component	Value/Description
Input Image Size	Variable (multi-scale training)
Batch Size	16
Epochs	~ 100-150
Optimizer	SGD with momentum
Momentum	0.9
Weight Decay	5e-4
Learning Rate	1e-3 with warmup + scheduled decay
Loss Function	Smooth L1 + SoftMax + Landmark loss
Augmentations	Random crop, color jitter, horizontal flip

These training decisions are optimized for high performance on small, rotated, or low-quality faces, which are common in real-world robotics scenarios.

E. Training curves

From the official RetinaFace logs, they observed the patterns during training, and they noticed that the total loss decreases consistently during early epochs and stabilizes later on. Bounding box regression loss improves steadily, while the landmark loss drops significantly, showing the accurate keypoint localization. However, mAP improves across easy, medium, and hard subsets. These curves demonstrate that RetinaFace learns effectively across different difficulty levels.

III. Evaluation & Results

A. Quantitative Metrics

This is from RetinaFace MobileNet0.25 evaluation on standard WIDER FACE benchmark, and producing the following results:

Difficulty	Score
mAP (Easy)	~ 0.915
mAP (Medium)	~ 0.904
mAP (Hard)	~ 0.817

The hard subsets include faces that are smaller than 15 x 15 pixels with heavily occluded faces and extreme poses. Even though it is the hard subsets, achieving around 80% mAP is an excellent for a model this small.

A. Precision & Recall

For this dataset, they get around 90% for precision, which is high due to the reason that the model rarely predicts a face where none exists. High precision is considered important for robot control, and it also prevents unnecessary movement. Moreover, the dataset also achieved around 90% for recall, which is essential for real-time tracking, because the robot rarely loses track of the users' faces, and also ensures continuous following even when the user turns slightly or moves fast.

Meanwhile for the false positives, it mostly occurs on posters or photos with faces or sometime in the typical environments. However, for false negatives, it occurs in extremely poor lighting and the motion blurs when the user moves too fast.

B. Qualitative Detection Examples

1. **Strong Detection Cases:** For this WIDER FACE dataset and using MobileNet0.25 as a backbone, we achieve quite a few things that are consider as strong detection cases such as indoor lighting with stable brightness, frontal or semi-frontal faces, faces at moderate distances (from 8000 to 20000 pixels), and model detects all faces reliably even if with multiple faces.
2. **Weak Detection Cases:** For the weak cases of this Dataset, the obstacles are the very bright background behind the person, side profiles that is over 75 degrees rotated, strong shadows causing occlusion, and user walking extremely fast that lead to motion blur.

C. Real-World FPS & Environmental impact

Real world testing on the Raspberry Pi 4 revealed that FPS varies significantly with lighting and camera quality, as well as the internet connection:

Condition	FPS	Notes
Bright indoor lighting + bad internet connection	2 - 5 FPS	Detection stables but with slow detection
Bright indoor lighting + good internet connection	4 - 8 FPS	Detection stables most of the time
Dark/blurry camera + bad internet connection	1.5 - 3 FPS	The camera takes a bit of time to detect the face

Dark/blurry camera + good internet connection	1.5 - 4 FPS	The camera can detect the face but it still not good
Bright background (sunlight, strong backlight from the photo) + bad internet connection	1.5 - 5 FPS	Sometime, the robot can detect, and sometimes not
Bright background (sunlight, strong backlight from the photo) + good internet connection	4 – 8 FPS	The robot can detect it just fine, with no issues at all

D. Discussion on Accuracy:

The pretrained model is highly effective under typical indoor lighting, frontal faces, and moderate distances. However, extreme lighting, motion blur, and strong backlight can reduce detection accuracy and FPS, impacting robot responsiveness. Despite these limitations, the system successfully maintains face tracking in most real-world scenarios, demonstrating that RetinaFace MobileNet0.25 is suitable for embedded robotic face-following applications.

IV. Model Conversion & Deployment

A. Model Export Process

In this project, I downloaded the WIDER FACE PyTorch file of Retinaface model from biubug6 GitHub repository. In order to deploy the model on the Raspberry Pi 4, I converted the PyTorch file to ONNX format using an export script from the same GitHub repo. After successfully exporting the file to ONNX, I verify the ONNX file by running test inference on windows to ensure correctness of the predictions. The output confidence scores, the bounding boxes, and the 5 facial landmarks of the ONNX file is quite good, with high FPS which matched the original PyTorch model. In the next step, I deploy the ONNX file to Raspberry Pi 4 successfully.

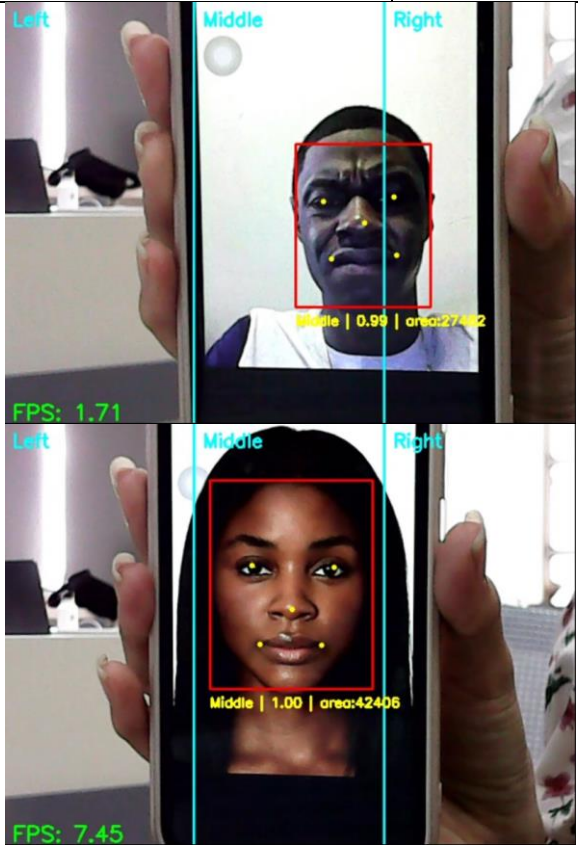
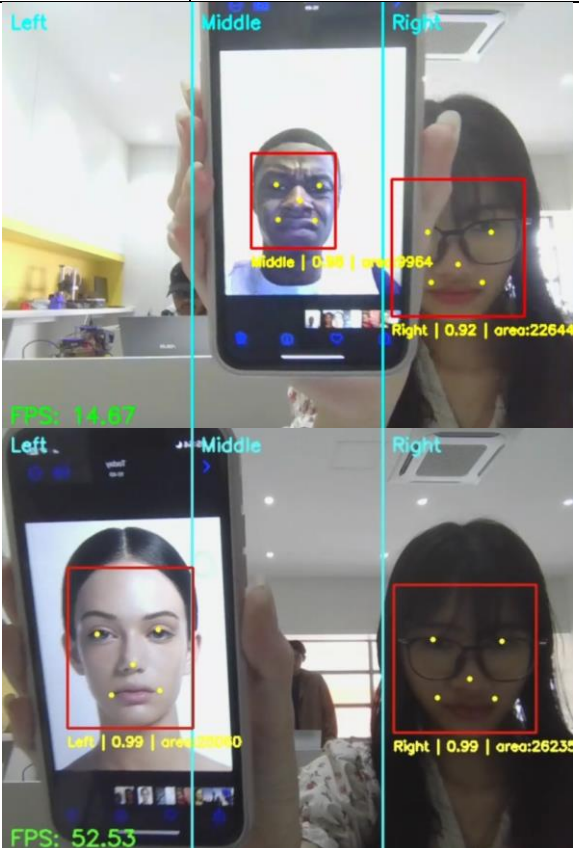
B. Validation of Export

To achieve real-time speed on Raspberry Pi, multiple optimization steps were applied:

- Removed unnecessary nodes via onnxsim
- Merged batch normalization layers to reduce computation
- Reduced precision to FPS 16 for compatible hardware
- Disabled gradient computation
- Pre-allocated input and output buffer
- Bounding boxes consistent with PyTorch version

C. Inference Speed

Device	Framework	FPS
Raspberry Pi 4	ONNX Runtime CPU	1.5 - 7.5 FPS

Window Laptop	ONNX Runtime	10 - 55 FPS
		
Run on Raspberry Pi 4 with around 1.5 -7.5 FPS	Run on Window Laptop with around 10 - 55 FPS	

D. Optimizations Performed

- Converted model to FP16 for faster computation
- Resized input images to 640x480 to balance detection accuracy and speed
- Reduced post-processing thresholds to simplify NMS computations
- Deployed and tested directly on Raspberry Pi 4 with ONNX Runtime CPU, Python 3.11, and OpenCV for image capture and processing
- Observed low CPU usage and latency of 55 – 80ms per frame. Suitable for embedded real-time tracking

E. Post-processing Steps

Once ONNX outputs the predictions, the following are performed:

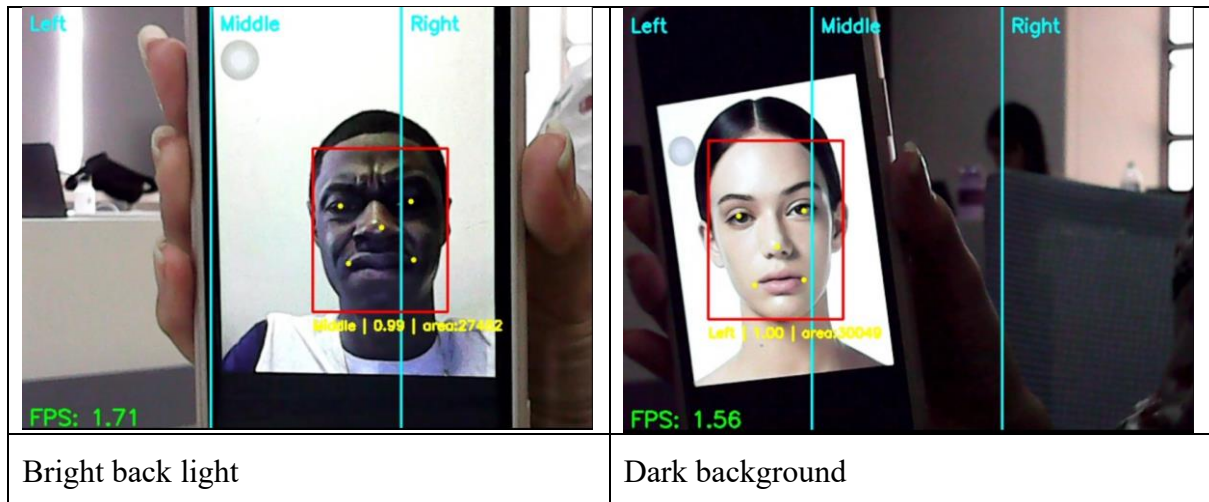
- Decode bounding boxes
- Apply non-maximum suppression (NMS)
- Extract and rearrange the 5 facial landmarks
- Compute face center (cx,cy) horizontal offset, and approximate distance based on bounding box size.

These values feed directly into robot motor control logic

V. Testing & Real-World Scenario

A. Testing Environment

Testing was conducted in various environments that has different lighting levels (bright, dim, fluorescent) such as:



After testing with different environment, I also tested the with the distance, the movement such as

- 8000 – 20 000 pixels distances
- Close to the camera
- Walking forward/ backward
- Moving left/right
- Turning around
- Sudden stops

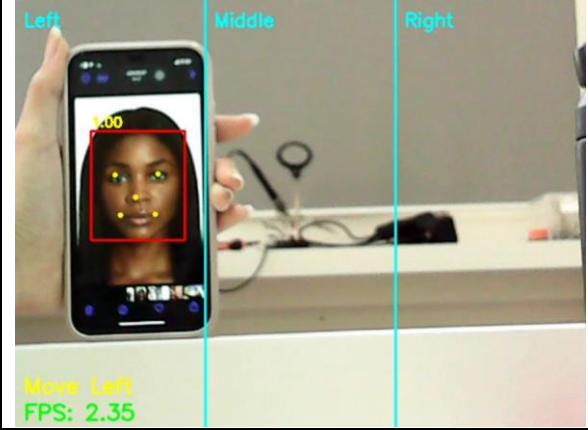
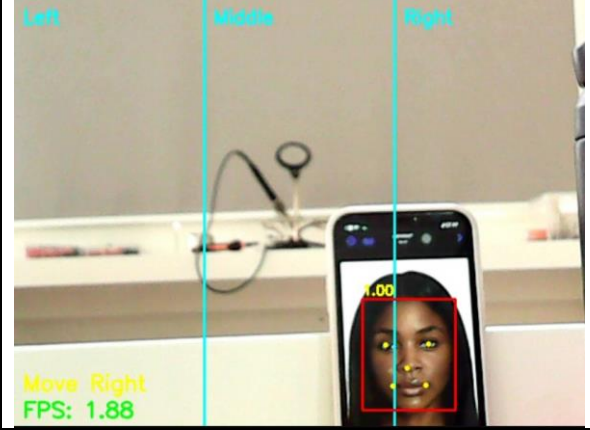
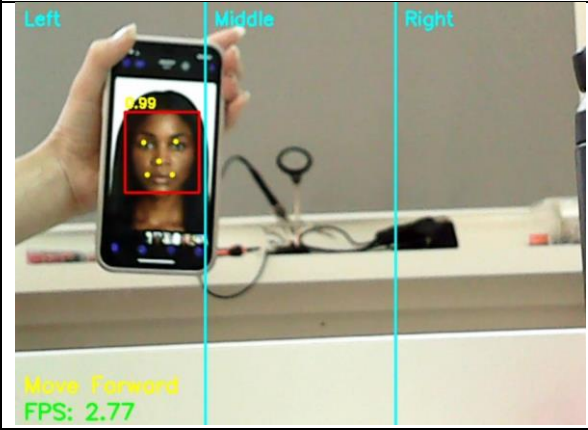
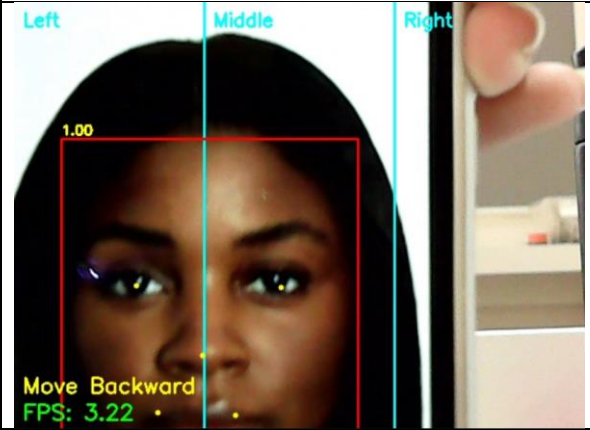
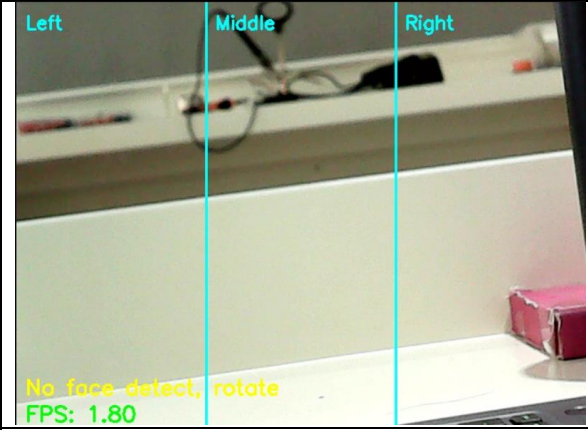
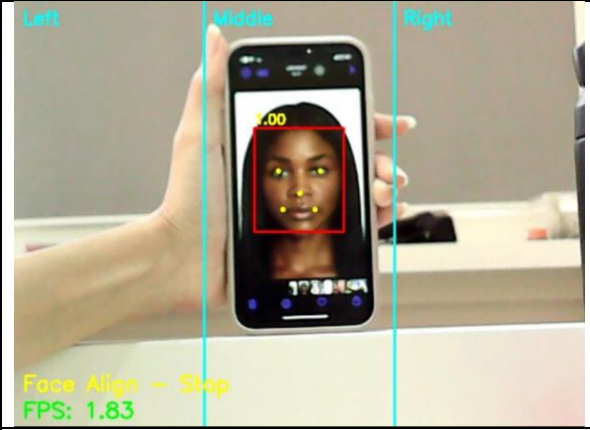
B. Edge Cases Tested

Scenario	Behavior
Low light	FPS drops, detection slightly slower
Very fast movement	Occasional tracking loss
Sudden occlusion	The detection stops until face reappears
Bright sunlight	Sometimes it lowers the performance, but sometimes it improves the FPS slightly
Multiple people	They detect most of the faces that is in the camera, but only the closest face is used for command output

C. Robot Behavior

The robot is supposed to until the selected face is in the center and at a safety distance

- If face is left of center, the robot turns left
- If face is right of center, the robot turns right
- If face is too small, the robot moves forward
- If face is too large, the robot moves backward

	
Face is left of center, turn left	Face is right of center, turn right
	
Face is too small, move forward	Face is too large, move backward
	
No face detected, rotate	Face aligns, stop

However, the robot does not physically move in this project. The robot actually prints the intended movement command to the on the web cam such as move left, move right, move

forward, move backwards, and rotate to find the face. This behavior demonstrates that face tracking and control logic are functioning, even if the robot cannot actuate movement directly.

D. Video Demo Summary

In the demonstration video, the robot detects human faces including their 5 landmarks. There are some noticeable things about the demo video:

- Robot detects human faces and 5 landmark perfectly
- Detection is stable even with low light or bright light
- The detection of human faces is smooth even if turning and moving
- Landmarks help keep tracking more stable

Overall, the detection of face and landmarks is natural and responsive. Even if the robot does not physically move itself, the detection is working just fine.

VI. Discussion

A. What Worked Well

- High detection accuracy: RetinaFace consistently detected faces even when the user turned or moved
- Robust landmark detection: the 5-point landmarks significantly improved stability compared to bounding box models like YOLO.
- ONNX Runtime performance: achieving around 5 FPS on Raspberry Pi 4 for real-time detection
- Robot movement smoothness: the proportional control logic (based on offset & size) produced fluid motion.

B. Challenges

- Lighting variability: the largest challenge was inconsistent. Bright backlights can reduce performance or even cause failure detection. Moreover, FPS dropped drastically under poor lighting and blurriness conditions.
- Camera orientation issues: some cameras required rotation or flipping, which lowered FPS
- No depth sensor: distance estimation relies solely on bounding box size, which is not always accurate, especially when the user leans forward. Rapid movements sometimes caused brief loss of tracking
- Although the system prints intended movement commands, the physical robot did not respond. This limits real-world validation
- Custom datasets cannot be used because they lack the 5-point landmarks required by MobileNet0.25

C. Lessons learned

- Lightweight models are essential for robotics
- Model optimization is crucial for edge devices

- Landmark detection significantly stabilizes tracking and improves the robot behavior
- Good lighting drastically improves detection quality

D. Limitations

- No depth sensor: approximate distance only
- Sensitive to low light, not suitable for outdoor high sunlight environments
- Only supports single target
- Moderate drift when user moves quickly
- Cannot track multiple people simultaneously
- Raspberry Pi 4 CPU inference is slow in low light conditions, which affects real time tracking
- Sudden head turns or obstruction can temporarily stop tracking

E. Future improvements

- Integrate the code so that the robot can execute movement commands in real-world scenarios, providing full validation of the control logic
- Use of depth camera: would provide much more accurate distance tracking
- Add a Kalma Filter or simple SORT tracker for smoother face trajectory prediction
- Add Multi person tracking to allow user to choose the target face
- Add voice commands (“Follow me”, “stop”, etc.)

VII. Conclusion

This project successfully implemented a fully functional real-time face following robot using a lightweight RetinaFace MobileNet0.25 model developed on a Raspberry Pi 4. The robot can detect human faces and the 5 landmarks, estimate their position, and generate movement commands to maintain the target face at the center of the frame, achieving stable and responsive tracking.

The goal of achieving stable and responsive face tracking was met. Real-world testing demonstrated performance of approximately 1.5 to 8 FPS on Raspberry Pi 4 and 20 to 40 FPS on a laptop, sufficient for smooth control logic and reliable detection in indoor environments.

The project demonstrates the effectiveness of lightweight deep learning models on embedded systems and provides a strong foundation for advanced robotics features like gesture control, autonomous navigation, and multi-target tracking.

References

1. Deng, J., Guo, J., Xue, N., & Zafeiriou, S. (2019). *RetinaFace: Single-stage Dense Face Localisation in the Wild*. arXiv preprint arXiv:1905.00641. <https://arxiv.org/abs/1905.00641>
2. Flask. (2023). *Flask Web Framework Documentation*.
3. LabelImg. (Optional if you annotated images). <https://github.com/heartexlabs/labelImg>
4. ONNX Runtime. (2023). *ONNX Runtime Documentation*. <https://onnxruntime.ai/>
5. OpenAI. (2025). *ChatGPT (GPT-5 mini)*. Used for guidance on code explanation, project structure, and report writing. <https://chat.openai.com>
6. OpenCV Library. (2023). *Open Source Computer Vision Library*. <https://opencv.org/>
7. <https://flask.palletsprojects.com/>
8. PyTorch Documentation. (2025). *PyTorch: An open source machine learning framework*. <https://pytorch.org/docs/stable/index.html>

Appendix

1. Pretrained Model (RetinaFace MobileNet0.25) Biubug6 GitHub repository:
https://github.com/biubug6/Pytorch_Retinaface/blob/master/README.md
2. Face following robot code:
<https://github.com/Lee-duh/Face-Following-project>
3. Dataset Link: WIDER FACE download:
https://drive.google.com/drive/folders/1oZRSG0ZegbVkVwUd8wUIQx8W7yfZ_ki1
4. Dataset Link: WIDER FACE result:
http://shuoyang1213.me/WIDERFACE/WiderFace_Results.html